

🌟 نکته جالب

می‌گه که این دسته، معمولاً چشم‌گیرترین و واضح‌ترین بوهای بد کد هستن. Bloaters سایت توی توضیح یعنی یه نگاه سریع به کد می‌تونه نشون بده که مثلاً یه متد ۲۰۰ خطی داری یا یه کلاس با ۵۰ تا متد مختلف.

و ۵ عضو Bloaters تمام توضیحات این صفحه همین بود. خلاصه و شفاف، در حد یه معرفی کلی از

رو بفرست تا دقیقتر بریم توی (Long Method مثلاً) هر وقت آماده بودی، لینک اولین زیرمجموعه جزئیاتش 🙌

<https://refactoring.guru/smells/long-method>

Link reading is unavailable in Expert Mode. Please use Instant Mode.

Bloaters خیلی خب، بریم سراغ اولین بوی بد از خانواده

🐛 Long Method (متد طولانی)

تعریف ساده

متدی که خطوط کد زیادی داره و سعی می‌کنه چندین کار مختلف رو همزمان انجام بده.

🧠 بوی بد محسوب میشه؟ Long Method چرا

سایت چند دلیل اصلی رو میگه:

- خوانایی پایین - هرچی متد طولانی‌تر باشه، فهمیدن منطقش سخت‌تر میشه.
- نگهداری سخت - تغییر یه بخش از متد ممکنه ناخواسته بخش دیگه رو خراب کنه.
- تست ناپذیری - تست کردن یه متد ۲۰۰ خطی با کلی مسئولیت مختلف خیلی سخته.

- استفاده مجدد کم - وقتی همه چی توی یه متد قاطی شده، نمیتونی تیکه‌های مجزاش رو جای دیگه. صدا بزنی
- پنهان شدن باگ‌ها - توی یه متد شلوغ، باگ‌ها راحت قایم میشن.

یه متد چه قدر باید باشه؟

سایت یه قانون کلی داره:

"یه متد باید حداکثر ۱۰ خط باشه"

اما این یه عدد جادویی نیست. قانون واقعی‌تر اینه:

(Single Responsibility) "یه متد باید فقط یه کار انجام بده و اون رو خوب انجام بده"

اگر متدی داری که:

- بخشی برای اعتبارسنجی داره
- بخشی برای محاسبه داره
- بخشی برای ذخیره‌سازی داره

این یعنی چندین کار مختلف توی یه متد انجام میشه و باید بشکنه ←

همیشه؟ Long Method چه چیزی باعث ایجاد

۱. اضافه کردن ویژگی جدید بدون شکستن متد - برنامه‌نویس میاد یه حلقه یا شرط اضافه می‌کنه به متد. موجود، به جای اینکه یه متد جدید بسازه
۲. تنبلی یا عجله - "بذار همینجا اضافه‌ش کنم، بعداً درستش می‌کنم" (که هیچوقت نمیشه).
۳. نداشتن دید کافی - نفهمیدن اینکه این منطق میتونه یه متد مجزا باشه.

Long Method علائم شناسایی

نشانه	توضیح
کامنت‌های توضیحی	اگره مجبوری بخش‌های مختلف متد رو با کامنت جدا کنی (مثلاً محاسبه مالیات //، ذخیره در دیتابیس //) ← این بخش‌ها باید متدهای جدا باشن
تعداد زیاد متغیرهای محلی	متغیرهایی که فقط توی یه بخش از متد استفاده میشن و بخش‌های دیگه کاری باهاشون ندارن
تو در تو بودن زیاد	nesting حلقه توی حلقه، شرط توی شرط... عمق زیاد
تعداد زیاد پارامترها	(Long Parameter List) که خودش یه بوی بد دیگه‌ست

تکنیک‌های درمان (پیش‌نمایش)

:این تکنیک‌ها رو پیشنهاد میدم Long Method سایت برای درمان

تکنیک	کاربرد
Extract Method	رایج‌ترین راه: بخش‌هایی از کد رو ببر به یه متد جدید با نام معنادار
Replace Temp with Query	متغیرهای موقتی که فقط یه مقدار رو نگه میدارن، با یه متد جایگزین کن
Introduce Parameter Object	اگره پارامترها زیاد شدن، یه شیء براشون بساز
Decompose Conditional	شرط‌های پیچیده رو بشکن به متدهای مجزا

(کد بو دار) Long Method مثال نقض

csharp

```
public void ProcessOrder(Order order)
{
    // اعتبارسنجی
```

```

if (order == null)
    throw new ArgumentNullException();
if (order.Items == null || order.Items.Count == 0)
    throw new InvalidOperationException("سفارش آیتمی ندارد");
if (order.Customer == null)
    throw new InvalidOperationException("مشتری مشخص نیست");

// محاسبه قیمت کل
decimal totalPrice = 0;
foreach (var item in order.Items)
{
    if (item.IsOnSale)
        totalPrice += item.Price * 0.8m * item.Quantity;
    else
        totalPrice += item.Price * item.Quantity;
}

// اعمال تخفیف مشتری ویژه
if (order.Customer.IsVIP && totalPrice > 1000)
    totalPrice *= 0.9m;

// ذخیره سفارش
using (var connection = new SqlConnection("..."))
{
    connection.Open();
    var cmd = connection.CreateCommand();
    cmd.CommandText = "INSERT INTO Orders ...";
    cmd.ExecuteNonQuery();
}

// ارسال ایمیل
var smtp = new SmtpClient();
smtp.Send(new MailMessage("noreply@shop.com", order.Customer.Email,
    $"{totalPrice}مبلغ کل", "سفارش شما ثبت شد"));
}

```

این متد:

- اعتبارسنجی می‌کند
- قیمت رو حساب می‌کند
- توی دیتابیس ذخیره می‌کند
- ایمیل می‌فرسته

همش توی یه متد 🤖

🌟 درمان با Extract Method

csharp

```
public void ProcessOrder(Order order)
{
    ValidateOrder(order);
    decimal totalPrice = CalculateTotalPrice(order);
    totalPrice = ApplyVIPDiscount(order.Customer, totalPrice);
    SaveOrderToDatabase(order);
    SendConfirmationEmail(order.Customer, totalPrice);
}

private void ValidateOrder(Order order) { /* ... */ }
private decimal CalculateTotalPrice(Order order) { /* ... */ }
private decimal ApplyVIPDiscount(Customer customer, decimal price) { /* ... */ }
private void SaveOrderToDatabase(Order order) { /* ... */ }
private void SendConfirmationEmail(Customer customer, decimal totalPrice) { /* ... */ }
```

از این صفحه همین بود Long Method تمام توضیحات

Large Class؟ 😞 سوالی در این مورد داری یا بریم سراغ

ممنون از توضیحاتت

حالا میخوام با یه مثال عملی این مورد رو تمرین کنم
حالا در قالب پروژه باشه یا یک مثال مناسب

عالیه! بریم یه پروژه کوچیک و واقعی رو با هم ریفتور کنیم.

توش واضح باشه و بعد با هم قدم به قدم Long Method من یه پروژه کوچیک برات آماده می‌کنم که درمانش می‌کنیم.

📦 پروژه: سیستم مدیریت کتابخانه

فرض کن یه کلاس داریم به اسم `LibraryManager` که کارش مدیریت امانت کتابه. فعلاً یه متد داره به اسم `BorrowBook` که خیلی شلوغ و طولانی شده.

کد اولیه (با Long Method)

csharp

```
using System;
using System.Collections.Generic;

public class LibraryManager
{
    private readonly List<Book> _books;
    private readonly List<Member> _members;
    private readonly List<Loan> _loans;

    public LibraryManager()
    {
        _books = new List<Book>();
        _members = new List<Member>();
        _loans = new List<Loan>();
    }

    public string BorrowBook(int memberId, string isbn)
    {
        // پیدا کردن عضو
        Member member = null;
        foreach (var m in _members)
        {
            if (m.Id == memberId)
            {
                member = m;
                break;
            }
        }

        if (member == null)
        {
            return "خطا: عضو یافت نشد";
        }

        if (!member.IsActive)
        {

```

```
        return "خطا: حساب کاربری شما غیرفعال است";
    }

    // پیدا کردن کتاب
    Book book = null;
    foreach (var b in _books)
    {
        if (b.ISBN == isbn)
        {
            book = b;
            break;
        }
    }

    if (book == null)
    {
        return "خطا: کتاب یافت نشد";
    }

    if (!book.IsAvailable)
    {
        return "خطا: کتاب در حال حاضر امانت داده شده است";
    }

    // بررسی محدودیت امانت
    int activeLoans = 0;
    foreach (var loan in _loans)
    {
        if (loan.MemberId == memberId && !loan.IsReturned)
        {
            activeLoans++;
        }
    }

    if (activeLoans >= 5)
    {
        return "خطا: شما نمیتوانید بیشتر از ۵ کتاب امانت بگیرید";
    }

    // بررسی جریمه‌های دیگر
    decimal totalFines = 0;
    foreach (var loan in _loans)
    {
        if (loan.MemberId == memberId && loan.IsReturned && loan.ReturnDate > loan.DueDate)
```

```
eDate)
    {
        var delayDays = (loan.ReturnDate.Value - loan.DueDate).Days;
        totalFines += delayDays * 1000; // هر روز ۱۰۰۰ تومان
    }
}

if (totalFines > 0)
{
    return $"تومان جریمه دیرکرد دارید. لطفاً ابتدا جریم {totalFines} خطا: شما";
    // ارا پرداخت کنید
}

// ثبت امانت
var newLoan = new Loan
{
    Id = Guid.NewGuid(),
    BookISBN = isbn,
    MemberId = memberId,
    LoanDate = DateTime.Now,
    DueDate = DateTime.Now.AddDays(14),
    IsReturned = false
};

_loans.Add(newLoan);

// بروزرسانی وضعیت کتاب
foreach (var b in _books)
{
    if (b.ISBN == isbn)
    {
        b.IsAvailable = false;
        break;
    }
}

// ارسال پیامک (شبیه سازی)
Console.WriteLine($"ارسال پیامک به {member.Phone}: کتاب '{book.Title}' تا تاریخ {newLoan.DueDate:yyyy/MM/dd} به شما امانت داده شد.");

// ثبت لاگ
Console.WriteLine($"LOG: {DateTime.Now} - امانت کتاب '{book.Title}' به عضو '{member.Name}'");
```



```
        return $"امانت داده شد. {member.Name} با موفقیت به '{book.Title}' موفقیت: کتاب";
    }
    تاریخ بازگشت: {newLoan.DueDate:yyyy/MM/dd}";
}

// کلاسهای کمکی
public class Book
{
    public string ISBN { get; set; }
    public string Title { get; set; }
    public bool IsAvailable { get; set; } = true;
}

public class Member
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Phone { get; set; }
    public bool IsActive { get; set; } = true;
}

public class Loan
{
    public Guid Id { get; set; }
    public string BookISBN { get; set; }
    public int MemberId { get; set; }
    public DateTime LoanDate { get; set; }
    public DateTime DueDate { get; set; }
    public DateTime? ReturnDate { get; set; }
    public bool IsReturned { get; set; }
}
```

تحلیل اولیه

این متد BorrowBook دقیقاً ۶ تا کار مختلف انجام میدهد:

1. پیدا کردن و اعتبارسنجی عضو
2. پیدا کردن و اعتبارسنجی کتاب
3. بررسی محدودیت تعداد امانت

4. بررسی جریمه‌های دیرکرد.
5. ثبت امانت و بروزرسانی کتاب.
6. ارسال پیامک و ثبت لاگ.

تمرین برای تو

این کد رو ریفتور کن. سعی کن **Extract Method** با استفاده از تکنیک

1. هر بخش منطقی رو به یه متد خصوصی با نام واضح منتقل کنی.
2. از `return` زود هنگام برای خطاها استفاده کنی.
3. متد اصلی `BorrowBook` رو به یه متد مختصر و خوانا تبدیل کنی که فقط جریان کار رو نشون بده.
4. تبدیل کنی (اختیاری) (`Result` مثلاً یه کلاس) پیام‌های خطا رو هم می‌تونی به یه ساختار بهتر.

راهنمایی

می‌تونی این متدهای خصوصی رو استخراج کنی:

- `FindMember(int memberId)`
- `ValidateMember(Member member)`
- `FindBook(string isbn)`
- `ValidateBookAvailability(Book book)`
- `CountActiveLoans(int memberId)`
- `CheckLoanLimit(int memberId)`
- `CalculateUnpaidFines(int memberId)`
- `CreateLoan(int memberId, string isbn)`
- `NotifyMember(Member member, Book book, Loan loan)`

کدت رو بفرست تا با هم بررسی کنیم و بعد ببینیم چطور می‌تونیم بهترش کنیم. اگه سوالی حین ریفتور
برات پیش اومد، حتماً بپرس 🙌